EchoFrame

First-Person Biographical Avatar Studio

Implementation Plans Across Three Deployment Contexts

An architectural planning document evaluating how an AI video-generation pipeline — with story-graph extraction, hybrid avatar construction, voice synthesis, lip-sync animation, GPU rendering, and C2PA provenance — would be structured, deployed, and operated across **Hostinger Horizons** (external SaaS), **Google Cloud Run** (internal variant), and **Poe.com** (embedded Canvas app).

Source repository	github.com/gracie007-cloud/EchoFrame
Application type	Async, multi-service AI video-generation pipeline
Compute profile	GPU-bound rendering; CPU-bound NLP, scripting, orchestration
Audience tiers	Students · creators · educators · museums · studios
Document type	Architectural planning artifact (no code, no mockups)
Prepared by	Web App Design Architect

Executive Summary

EchoFrame (also branded *AethelFrame* in the README) is an AI-powered video-generation platform that produces cinematic, first-person biographical documentaries narrated by dynamically animated avatars of the subject. Subjects span humans, animals, mythological figures, fictional characters, and even sentient artifacts. The pipeline is end-to-end: raw input is transformed into a structured story graph, then a script, then a hybrid anthropomorphic avatar, then synthesized voice, then lip-synced animation, then a fully rendered video carrying C2PA provenance metadata.

The defining architectural fact

Unlike a typical web app, EchoFrame has a **real, GPU-bound backend**. The repository structure makes this explicit — there are directories for a Python SDK, an async-job-state webhook specification, a webhook-bridge microservice, a compliance/provenance architecture, and a story-graph service. This is a distributed system, not a single page application.

The implication for deployment: none of the three target contexts can host EchoFrame end-to-end on its own. Each plan below describes which slice of the pipeline that platform is suited to host, what it must delegate, and how that delegation should be structured. The final section of this document presents a layered composition view showing how the three contexts can — and arguably should — operate together rather than as alternatives.

What this document delivers

- A concept summary anchored in the actual repository structure and pipeline stages
- Three implementation plans, each honest about what the platform can host versus delegate
- A side-by-side comparison matrix across eleven architectural dimensions
- A layered composition view showing how the three contexts can serve different funnel stages of the same product
- Recommendations and next steps for sequencing the deployments

What this document does not include

- Source code, snippets, or executable assets
- Visual mockups, wireframes, or system diagrams (the prose describes architecture; rendered diagrams are out of scope for this artifact)
- Vendor pricing breakdowns or commercial procurement guidance
- Step-by-step deployment runbooks

Concept At A Glance

The pipeline, stage by stage

The README describes a deterministic, cloud-optimized pipeline. Translated into architectural stages, each with its own compute and storage profile:

Stage	What it does	Compute profile
1. Input ingestion	Accepts text, URL, audio, or images	Light · CPU
2. NLP & story graph	Extracts entities, dates, causal chains; builds three-act arc	Moderate · CPU + LLM call
3. Script generation	LLM produces editable narrative; tags lines as Verified, Speculative, Fictional	Low · External API
4. Avatar construction	Hybrid: trait-mapping onto humanoid rig; FLUX/SDXL → ControlNet → TripoSR/NeRF	Heavy · GPU
5. Voice synthesis	TTS plus harmonic layering for non-human timbres; consent-gated cloning	Moderate · GPU
6. Animation & lip-sync	LivePortrait, Wav2Lip++, IK gestures, micro-expression smoothing	Heavy · GPU
7. Scene composition	HDRI environments, B-roll, archival overlays	Moderate · GPU
8. Render & provenance	Final video render with embedded C2PA metadata	Heavy · GPU + storage
9. Export & delivery	MP4, web-optimized variants, VTT captions	Light · CPU + CDN

What the repository tells us about engineering intent

The repo has thirteen top-level directories. Five of them — **async-job-state-webhook-specification**, **webhook-bridge-microservice**, **python-SDK**, **SDK-wrappers**, and **compliance-provenance-architecture** — are signals of a system designed for *API consumers*, not just an end-user UI. EchoFrame is being engineered as a platform with multiple front-doors: a GUI dashboard, a developer CLI, language-specific SDKs, and webhook-driven integrations. This shapes every deployment decision below.

Cross-cutting concerns inherited by every deployment

Three concerns recur in the README and must be addressed wherever the system lives:

- **Async by nature** — full-quality renders cannot complete inside a synchronous HTTP request. Webhooks and job-state polling are first-class, not afterthoughts.
- **Compliance by default** — C2PA provenance, consent verification for living-person voice/likeness, and the cultural-sensitivity advisory layer are part of the pipeline, not optional add-ons.
- **Tiered output quality** — Starter (30s, watermarked), Creator (3min, HD), Studio (10min+, priority), Enterprise (batch, white-label). Different tiers want different infrastructure shapes.

Note on naming: the repo title is ‘EchoFrame’ but the README uses ‘AethelFrame’ throughout. This document uses **EchoFrame** as the canonical name; one of these should be retired before any public deployment to avoid customer confusion.

Implementation Plan A: External SaaS on Hostinger Horizons

Public commercial surface · creator and educator tiers · control plane only

Context fit and honest constraints

Hostinger Horizons is a no-code, prompt-driven SaaS builder optimized for shipping monetizable web products quickly. It handles auth, hosting, basic data persistence, billing, and a marketing surface — all the operational scaffolding a creator-tier SaaS needs. **What it does not do** is run heavy GPU workloads, host Python ML pipelines, or execute multi-minute background jobs. That is fine, because EchoFrame's render pipeline already expects to be called over a network as an async job.

> **Key architectural decision:** Horizons is the *control plane and presentation layer* only. The render pipeline lives elsewhere — typically the Cloud Run deployment described in Plan B, or a third-party render farm. Horizons submits jobs and receives results; it does not produce video itself.

Architecture overview

- **Marketing surface** — landing pages, pricing, blog, FAQ, public ethics and provenance documentation; all SEO-indexable
- **Account & billing** — Horizons-managed users, Starter/Creator/Studio tier gating, credit balance display, subscription management
- **Submission UI** — input forms (text, URL, audio upload), tone preset selection, avatar style chooser, length and quality options
- **Script editor** — read/edit the LLM-generated script, accept or reject accuracy tags, approve before render
- **Job dashboard** — list of in-flight and completed renders, status indicators, queue position, ETA
- **Output gallery** — finished videos with download, share, re-render, and version history
- **Webhook receiver endpoint** — paired with the EchoFrame webhook-bridge microservice; receives job-state updates from the render backend and updates the dashboard in real time

Authentication and user management

Horizons-native email and social login. Tiered feature gating maps directly to the monetization model in the README: Starter (free, watermarked, 30s, public-domain only), Creator ($19/mo), Studio ($49/mo), Enterprise (custom, hand-off to Plan B). Consent verification for voice/likeness submissions runs as a Horizons workflow with cryptographic attestation stored against the user account.

Data flow for a single render job

User submits inputs in Horizons UI → Horizons posts a job to the EchoFrame render API (a Cloud Run endpoint, an external GPU service, or both) → render backend issues a job ID → Horizons stores the job ID against the user, polls or awaits a webhook, displays progress → render backend pushes state

updates (queued, scripting, rendering, done, failed) → Horizons updates the dashboard, notifies the user, presents the finished video for download.

Operational considerations

- **API key management** — credentials for the render backend live in Horizons' secret store; never exposed to the browser
- **Webhook security** — incoming webhooks must be signed and verified; replay protection is required since the README explicitly designs an async webhook spec
- **Storage of finished videos** — Horizons-hosted is fine for short-term download; long-term archival belongs in object storage (S3, GCS) with signed URLs surfaced through the dashboard
- **Provenance display** — C2PA metadata must be surfaced in the UI as a verifiable badge, not just embedded in the file; this is a trust feature, not a technical detail
- **Watermarking** — Starter-tier watermark must be applied at render time, not by Horizons; Horizons only enforces tier policy at job-submission

Trade-offs

Strengths: fastest path to a paying customer; Horizons absorbs auth, billing, marketing, and CDN concerns; the no-code iteration loop is well-suited to A/B testing the submission UI and pricing page. **Limits:** Horizons is not the system — it cannot exist without a render backend; vendor lock-in is real; advanced UI requirements like the story-graph visual editor or the script timeline editor may strain Horizons' component model and may need to live in a separate embedded micro-frontend.

Implementation Plan B: Internal Variant on Google Cloud Run

Full pipeline · enterprise tier · governance, GPUs, and audit by default

Context fit (with a 2025 update worth flagging)

Cloud Run is the strongest of the three contexts for EchoFrame because — **as of June 2025** — it offers generally available NVIDIA GPU support: L4 (24 GB VRAM) and RTX PRO 6000 Blackwell (96 GB VRAM), available across services, jobs, and worker pools. Combined with scale-to-zero, per-second billing, and Cloud Run Jobs (which support GPU and have no 60-minute request timeout), this is enough infrastructure to host the entire EchoFrame pipeline rather than just a slice of it.

■ **Worth verifying at build time:** Cloud Run GPUs are currently available in five regions (us-central1, europe-west1, europe-west4, asia-southeast1, asia-south1). Confirm region availability against your customers' data residency requirements before committing. Pricing as of late 2025 is roughly $0.67/hour for an L4 without zonal redundancy — the cost envelope is real but predictable.

Architecture overview — the system, not a slice

Cloud Run lets EchoFrame run as a small constellation of services and jobs, each scaling independently. A reasonable decomposition follows the repository's own structure:

- **API gateway service** (Cloud Run service, CPU only) — receives job submissions, validates inputs, returns job IDs, exposes the SDK contract documented in the python-SDK directory
- **Story-graph service** (Cloud Run service, CPU + LLM external call) — runs the NLP pipeline and returns the graph representation that drives script generation
- **Script service** (Cloud Run service, CPU + LLM external call) — generates and tags the narrative; supports the editable-script workflow
- **Render workers** (Cloud Run jobs with GPU attached) — long-running jobs for avatar construction, voice synthesis, lip-sync, and final composition; L4 for most work, RTX PRO 6000 for the largest models or longest renders
- **Webhook bridge** (Cloud Run service, CPU only) — emits signed job-state events to consumers (Plan A's Horizons UI, third-party integrations); maps directly to the webhook-bridge-microservice directory in the repo
- **Provenance service** (Cloud Run service, CPU only) — embeds C2PA metadata, generates consent receipts, runs the IP-scanning checks; this is where the compliance-provenance-architecture directory lives
- **Persistent stores** — Cloud SQL for relational data (users, jobs, accounts), Firestore for job-state events and audit logs, Cloud Storage for video artefacts and intermediate render assets

Authentication and user management

For internal/enterprise use, Identity-Aware Proxy fronting the API gateway is the simplest and most defensible pattern. Workspace SSO covers internal teams; service accounts cover machine-to-machine traffic from the Plan A SaaS or third-party SDKs. For external API consumers, signed JWTs issued at account provisioning, scoped to tier and quota, validated at the gateway. Roles map cleanly to the README's monetization tiers and to the internal-app personas described in DESIGN.md.

Why Cloud Run jobs (not services) for rendering

Cloud Run services are request-driven and capped at 60 minutes per request. A full Studio-tier render of a ten-minute documentary will exceed that. Cloud Run *jobs* are explicitly designed for batch and async work, support GPUs, and have no per-invocation timeout in the same way. The architecture should run the API gateway as a service (responsive, low-latency, scales-to-zero between requests) and dispatch render workloads as jobs (heavy, GPU-bound, scales independently of API traffic). This split is the single most important design decision in Plan B.

Operational considerations

- **Audit trail** — every render job logged with submitter identity, input snapshot, model versions, output hash, and provenance receipt; this is non-negotiable for the museum and publisher tier
- **Secrets handling** — model weights, third-party API keys, and consent attestation private keys live in Secret Manager, never in container images
- **Cost shape** — scale-to-zero on every service plus on-demand GPU jobs means cost tracks usage closely; the dominant cost variable is GPU-hours, which makes the per-tier render limits in the README directly enforceable as quota
- **Observability** — Cloud Logging structured logs, Cloud Monitoring metrics on job duration and queue depth, Cloud Trace for cross-service request paths; this is the platform's native stack and should be used end-to-end
- **CI/CD** — Cloud Build or GitHub Actions, image scanning, staged rollouts, instant rollback to prior revisions; this is the deployment model when governance matters
- **Cold starts** — GPU instances start in roughly 5 seconds plus model-load time; for interactive tier (Starter previews) keep at least one warm instance via min-instances=1; for batch tier (Studio, Enterprise) cold start is acceptable

Trade-offs

Strengths: the only context that can host the full pipeline; production-grade governance, audit, and observability; Cloud Run GPUs make this commercially viable in a way it was not twelve months ago; clean separation of services and jobs matches the system's natural shape. **Limits:** highest engineering investment of the three plans; requires container, GPU, and orchestration expertise; not a marketing surface — Cloud Run hosts services, not websites; quota and region availability for GPUs needs early verification.

Implementation Plan C: Embedded Canvas App on Poe.com

Top of funnel · scoped teaser · script and concept only

Context fit and the honest scope question

Poe Canvas apps run in a sandboxed environment optimized for compact, prompt-driven tools. They cannot do GPU rendering. They cannot persist multi-minute background jobs. They cannot return MP4 video as a Canvas output. **The full EchoFrame pipeline cannot run on Poe.** What Poe can do — and do well — is host a scoped *slice* of the experience that is genuinely useful on its own and serves as a discovery vehicle for the larger product.

The right slice: script and concept generation

Three of EchoFrame's nine pipeline stages are LLM-bound rather than GPU-bound: story-graph extraction, script generation with accuracy tagging, and tone-and-pacing preview. These are precisely the stages Poe runs natively, against the model the user is already paying for. The Canvas app does not attempt to render video; it produces the *narrative blueprint and avatar concept* the user would carry to Plan A or Plan B for the full render.

Architecture overview

- **Single Canvas surface** with three sequential modes: input → story graph → editable script with concept
- **Input form** — subject name, era or origin, source material (text/URL), tone preset, target length, avatar style preference
- **Story graph view** — visual rendering of entities, dates, and the three-act arc; users can edit, drop, or reorder beats before script generation
- **Script panel** — full first-person narrative, with each line tagged Verified, Speculative, or Fictionalized; users can edit inline
- **Avatar concept thumbnail** — a single still rendered through Poe's native image model based on the trait-mapping description; not animated, not lip-synced, but visually concrete
- **Hand-off action** — 'Render this in EchoFrame Studio' button that exports the script and concept as a structured payload (JSON) the user can carry to the SaaS or feed to the Python SDK

Authentication and user management

Delegated entirely to Poe. The Canvas app inherits whoever is signed in — there is no separate account system. This is a real simplification but a constraint: the app cannot enforce custom roles, tiers, or feature gating beyond what Poe's subscription model provides. For a top-of-funnel discovery surface, that is acceptable; for a primary commercial product, it would not be.

Why this is the strongest top-of-funnel surface

Poe’s discovery model — users browsing the app directory, conversational entry, frictionless authentication — is precisely how this kind of niche creative tool finds its audience. The script-and-concept slice is genuinely useful even if the user never proceeds to a full render: writers, educators, and museum curators can use it as a research and drafting tool. That standalone utility is what makes it a credible funnel entry point rather than a thin teaser.

Operational considerations

- **Scope discipline** — ship the script-and-concept tool only; no video, no animation, no audio synthesis in this surface
- **Compliance carry-over** — the consent and IP-scanning gates from the full pipeline still apply at the Canvas stage; do not generate scripts for living non-public-figure individuals without explicit attestation
- **Hand-off integrity** — the JSON export must be schema-stable so the Plan A SaaS and the Python SDK can consume it without renegotiation; this maps to the schema directory in the repository
- **Persistence trade** — no draft saves beyond Poe’s chat history; users who want to return to a draft must export it
- **Distribution** — Poe’s creator economy is the main monetization channel for this surface; pricing for the full render still belongs in Plan A

Trade-offs

Strengths: lowest operational burden of the three plans; built-in distribution; uses Poe’s LLM directly for the parts of the pipeline best suited to it; provides real value standalone, not just as a demo. **Limits:** cannot produce the actual video the product is named for; sandbox prevents GPU work; no custom auth, billing, or feature gating beyond Poe’s subscription model; ephemeral by design.

Comparison Matrix

Eleven architectural dimensions, three deployment contexts. Read the matrix as a decision aid that frames trade-offs rather than ranks options — each platform optimizes for a different stage of the product's lifecycle.

Dimension	Hostinger Horizons	Google Cloud Run	Poe Canvas
Hosts full pipeline?	No · control plane only	Yes · entire system	No · script/concept only
GPU support	None	L4 (24GB), RTX PRO 6000 (96GB)	None
Async render jobs	Delegated to backend	Native (Cloud Run jobs)	Out of scope
Authentication	Built-in (email, social)	IAP / Workspace SSO / JWT	Delegated to Poe
Persistence	Platform-managed	Cloud SQL + Firestore + GCS	Chat history only
Provenance & audit	Display only	Native, end-to-end	Not applicable
Compliance enforcement	At submission	At every stage	At input only
Best audience tier	Creator, Studio	Studio, Enterprise	Starter, discovery
Deployment effort	Lowest	Highest	Low
Primary risk	Vendor lock-in	GPU cost & quota	Limited scope

Layered Composition: Three Contexts, One Product

Because no single context can host EchoFrame end-to-end, the three plans are best treated as **complementary surfaces of the same product** rather than alternatives. The following composition is the strongest articulation of how they fit together — each platform serves a different funnel stage and a different audience tier, all backed by the same render core.

Funnel mapping

Stage	Surface	Audience	Output
Discovery	Poe Canvas	Browsers, hobbyists, researchers	Script + avatar concept
Conversion	Hostinger Horizons	Creators, educators, indie producers	Full rendered video, watermarked or licensed
Operationalize	Google Cloud Run	Museums, studios, publishers, internal teams	Audited renders, batch outputs, API access

How a single user moves across the three

A history teacher discovers EchoFrame through Poe’s app directory while researching a lesson plan. They generate a script and avatar concept for Hatshepsut, edit the tone, and export. The Canvas hand-off button carries the JSON payload to the EchoFrame SaaS on Horizons, where they sign up for the Creator tier and render a three-minute documentary with HD output and accuracy tags preserved. A year later, their school district licenses the platform across forty classrooms and is provisioned through the Cloud Run-hosted Enterprise tier — same script schema, same avatar concept, same render core, with batch rendering, audit trails, and white-label export now in scope.

What this requires from the engineering team

- **A stable schema** shared across all three surfaces — the schema directory in the repository is doing exactly the right work; it is the contract that lets Canvas hand off to Horizons hand off to Cloud Run without renegotiation
- **A single render core** — the Cloud Run jobs from Plan B serve all three surfaces; Horizons and Canvas are clients of it, not competitors to it
- **Consistent provenance** — C2PA metadata applied at the Cloud Run render stage means every video carries the same trust signal regardless of which surface initiated the job
- **Aligned tier semantics** — Starter/Creator/Studio/Enterprise must mean the same thing across all three surfaces; this is product policy work, not platform work

Recommendations and Next Steps

If you have to pick one to build first

Build Plan B (Cloud Run) first. It is the only context that hosts the actual video pipeline. Without it, the other two plans have nothing to delegate to — Horizons becomes a marketing site for a product that does not exist, and Poe becomes a tool that produces scripts users cannot turn into videos. Get the render core working end-to-end on Cloud Run with one or two avatar styles, validate the pricing envelope against real GPU costs, and expose a stable API.

Sequencing recommendation

- **Phase 1 (months 0–4) — Cloud Run MVP.** API gateway, story graph, script service, one avatar pipeline, render jobs on L4 GPUs, webhook bridge, C2PA embedding. Internal use only, gated by IAP. Validate the pipeline produces watchable video at acceptable cost per minute.
- **Phase 2 (months 4–7) — Hostinger Horizons SaaS launch.** Stand up the public-facing creator tier on top of the Cloud Run backend. Marketing site, accounts, billing, submission UI, dashboard, output gallery. Starter and Creator tiers go live; Studio gated by manual approval until pipeline reliability is proven.
- **Phase 3 (months 7–10) — Poe Canvas top-of-funnel.** Ship the script-and-concept slice on Poe with the hand-off path to Horizons. Use the Canvas app as cheap, distributed user research — the inputs people actually fill in there are the inputs you should optimize the SaaS forms for.
- **Phase 4 (months 10–12) — Cloud Run Enterprise tier.** White-label deployments, batch rendering, museum and studio licensing flows; this is where the compliance-provenance-architecture and SDK-wrappers directories pay back their investment.

Cross-context fixes worth doing once

- **Resolve the EchoFrame / AethelFrame name conflict** before any public surface ships; pick one and update the README, repo description, and any in-product copy
- **Lock the schema** in the repository's schema directory before building hand-off paths between surfaces; schema drift across three deployments is the single biggest preventable failure mode
- **Define tier semantics in product, not platform** — Starter/Creator/Studio/Enterprise should be a shared dictionary that Horizons, Cloud Run, and Canvas all read from, not three independently coded interpretations
- **Stand up the consent and IP-scanning gates early** — they are mentioned in three of the README's frameworks (copyright, consent, cultural sensitivity) and are far cheaper to design before three surfaces depend on them than to retrofit
- **Verify Cloud Run GPU region availability** against your earliest expected enterprise customer's data residency requirements; this is the single most likely architectural blocker

What to validate before committing

- **End-to-end cost per render minute** — get a real number on L4 GPU time, LLM API calls, storage, and bandwidth before pricing tiers are locked in; the README's prices are aspirations until this is measured
- **Render reliability at the longest tier length** — Studio tier promises 10-minute renders; the temporal-drift mitigation in the roadmap is a real technical risk and should be proven on Cloud Run before being marketed
- **The hand-off integrity loop** — generate a script in Canvas, carry the JSON to Horizons, render in Cloud Run, verify the output matches the user's intent across all three surfaces; do this manually ten times before automating it

---

End of document. No code, no mockups, no follow-up questions required.